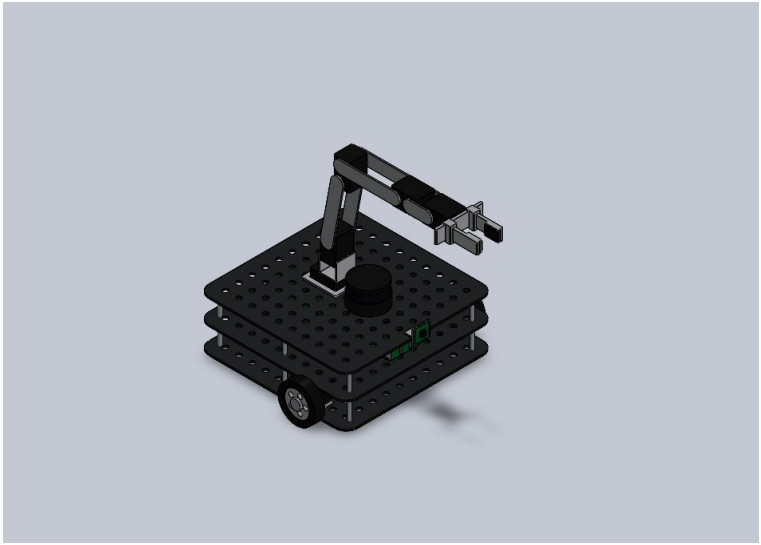
PROJECT REVIEW REPORT

Motion Planning for a TurtleBot3 + OpenMANIPULATOR-X Mobile Manipulator: A Comparison of RRT, RRT* and PRM



Subject: Robot Dynamic Control

Team members

Kaviyashree T S

Barath K M

Jyothish S

Nabeeha Naaz H

Project review: 14 October 2026

1. Abstract

A mobile manipulator is a wheeled robot with an arm on top, and the arm must move around obstacles without touching them. In this project we built a model, ready for simulation, of a TurtleBot3 Waffle Pi carrying an OpenMANIPULATOR-X arm (4 joints plus a two-finger gripper) and used it to study three sampling-based motion-planning algorithms: RRT, RRT* and PRM. The robot was modelled in SolidWorks 2025 as 19 parametric parts and 2 assemblies, exported as a STEP file and converted into a ROS 2 Humble package (URDF/xacro with STL meshes) whose joints follow the official ROBOTIS description. To study the algorithms we wrote the three planners in Python on the arm's kinematic chain and tested them in a scene where the straight path from start to goal is blocked, with 10 random runs per planner. All three succeeded every time. RRT was the fastest (about 43 ms) but its paths were long (4.32 rad on average). RRT* took about 87 ms for its first path and, given a 10 s budget, improved it to 2.83 rad, which is 18 % above the straight-line lower bound of 2.40 rad. PRM needed about 10.7 s to build its roadmap and returned paths of 3.64 rad, but the roadmap can be reused for later queries. The full MoveIt 2 / OMPL / Gazebo comparison is the next step.

Keywords: mobile manipulator, TurtleBot3, OpenMANIPULATOR-X, motion planning, RRT, RRT, PRM, MoveIt 2, OMPL, ROS 2, SolidWorks.*

2. Background

A mobile manipulator can drive to a location and then work on objects there, which is useful for fetching, sorting, inspection and service tasks. The TurtleBot3 Waffle Pi is a popular, low-cost ROS 2 platform with a differential drive, an LDS-01 LiDAR, a Raspberry Pi computer, an OpenCR controller and a camera. The OpenMANIPULATOR-X is a 4-DOF arm with a gripper, built from DYNAMIXEL XM430-W210 servos. ROBOTIS offers the two together as the TurtleBot3 Manipulation setup.

Moving the arm safely is a motion-planning problem. We need a smooth path in joint space from a start pose to a goal pose that stays clear of obstacles and of the robot's own body. Every extra joint multiplies the number of possible poses, so checking them all is not practical. Sampling-based planners get around this by testing random poses, and they are what MoveIt 2 and the OMPL library use in ROS 2.

To test planners in simulation we first need a trustworthy 3-D model of the robot. The choice of planner (RRT, RRT* or PRM) also matters: it changes how long planning takes, how good the path is, and whether the work can be reused. So the project has three parts: (i) build an editable

parametric model that matches the real robot's kinematics, (ii) package it for ROS 2, MoveIt 2 and Gazebo, and (iii) study and compare the three planning algorithms.

2.1 Objectives

1. Build a parametric SolidWorks 2025 model of the TurtleBot3 Waffle Pi with the OpenMANIPULATOR-X, including working revolute joints and a two-finger gripper.
2. Export it as a STEP file and as a ROS 2 Humble description package (URDF, xacro, STL meshes, RViz launch).
3. Check that the kinematics, joint limits and inertial values match the official ROBOTIS model.
4. Explain RRT, RRT* and PRM mathematically, simulate each one on the robot and justify its behaviour.
5. Compare the three on planning time, path length, success rate and node count.
6. Get the package ready for the MoveIt 2 + OMPL + Gazebo comparison.

3. Literature Review

Table 1 lists the work this project builds on. For each source we note what it contributed (key findings) and where it falls short for our purpose (limitations / research gap).

No.	Title	Key findings	Limitations / Research gap
1	**LaValle (1998)** Rapidly-exploring random trees: a new tool for path planning [1]	Introduced RRT: a tree grown by sampling random states and extending the nearest node. Probabilistically complete, works in high-dimensional spaces, very quick to a first path.	Paths are jagged and not optimal; behaviour depends on step size and distance metric.
2	**Kavraki et al. (1996)** Probabilistic roadmaps for path planning in high-dimensional configuration spaces [2]	Introduced PRM: build a roadmap of free configurations once, then answer many queries by graph search. Good for static scenes and many goals.	Roadmap construction is costly; narrow passages are hard; roadmap must be rebuilt if obstacles move.
3	**Kuffner & LaValle (2000)** RRT-Connect: an efficient approach to single-query path planning [4]	Grows two trees (from start and goal) that try to connect greedily; much faster than basic RRT on manipulator-type problems.	Still not optimal, so paths need post-smoothing; only for problems where two states can be connected directly.
4	**Karaman & Frazzoli (2011)** Sampling-based algorithms for optimal motion planning [3]	Showed that RRT and PRM are not asymptotically optimal; proposed RRT* and PRM*, which converge to the optimal path with the same asymptotic complexity.	Convergence can be slow in higher dimensions; extra cost per sample from neighbour search and rewiring; radius needs tuning.
5	**Gammell et al. (2014)** Informed RRT* [13]	After the first solution, samples only inside an ellipsoid that can still improve it, so the path converges faster than RRT*.	Needs a first solution; advantage shrinks as dimension grows; assumes a simple distance-based cost.
6	**Sucan et al. (2012)**	Common C++ interface to many planners (RRT, RRT*, PRM and more)	Generic library: results depend on the robot model, collision checker and

	The Open Motion Planning Library [5]	with benchmarking tools; it is the planning back-end of MoveIt.	parameters chosen by the user.
7	**Chitta et al. (2012)** MoveIt! [6]	Brings kinematics, collision checking, planning and execution together for manipulators in ROS.	Needs per-robot configuration; arm-centred, so planning base and arm together takes extra set-up.
8	**Macenski et al. (2022)** Robot Operating System 2 [7]	Describes ROS 2 design: DDS communication, real-time support, multi-robot use and security.	General framework with no manipulation-specific planning comparison.
9	**Koenig & Howard (2004)** Design and use paradigms for Gazebo [8]	Open-source simulator with physics and sensor models, widely used to test robots before hardware.	Simulation-to-real gap; results are only as good as the URDF, inertia and friction values supplied.
10	**ROBOTIS (e-Manual, GitHub) [11], [12]** turtlebot3_manipulation	Official URDF, MoveIt configuration and Gazebo files for the TurtleBot3 + OpenMANIPULATOR-X.	No editable parametric CAD source and no planner comparison; meshes are fixed.

Table 1. Literature summary.

3.1 Research gap

- Many planner studies use fixed-base industrial arms rather than a small ROS 2 mobile manipulator.
- Papers compare algorithms in general, but give no editable CAD-to-URDF model of the TurtleBot3 + OpenMANIPULATOR-X to repeat the study on.
- The official ROBOTIS package gives the model and a working MoveIt setup, but no side-by-side RRT / RRT* / PRM comparison and no parametric CAD.
- RRT, RRT* and PRM are often explained through theory and generic benchmarks; we also show each one running on the same robot with the same metrics.

3.2 Novelty of this work

To the best of our knowledge, and based on the sources above, the contributions of this project are:

1. One specification file (robot_spec.py) generates both the SolidWorks model and the URDF, so geometry and kinematics cannot drift apart. Changing one parameter, e.g. forearm length, updates the CAD and the simulation model together.
2. A fully parametric SolidWorks 2025 model of the TurtleBot3 Waffle Pi + OpenMANIPULATOR-X (19 parts, working joint mates, two-finger gripper) with a drop-in ROS 2 package for MoveIt 2 and Gazebo.
3. A like-for-like comparison of RRT, RRT* and PRM on this robot: same kinematics, same collision model, same start/goal and same metrics, with a simulation and a written justification for each algorithm.
4. A reproducible benchmark script that the team can re-run, and a clear path to repeat the same study in MoveIt 2 / OMPL / Gazebo.

4. Methodology

4.1 System architecture

The project has four layers (Figure 1). The design layer holds the single specification file (robot_spec.py) and the SolidWorks model, and produces the STEP file and renders. The description layer turns the same specification into a URDF/xacro package with STL meshes that opens in RViz. These two layers are finished. The planning/control layer (MoveIt 2 with OMPL planners and ros2_control controllers) and the simulation/hardware layer (Gazebo, Nav2, the real robot) are the next step, and the package is already prepared for them: link and joint names match the official ROBOTIS files, so the official SRDF, controllers and ompl_planning.yaml work unchanged.

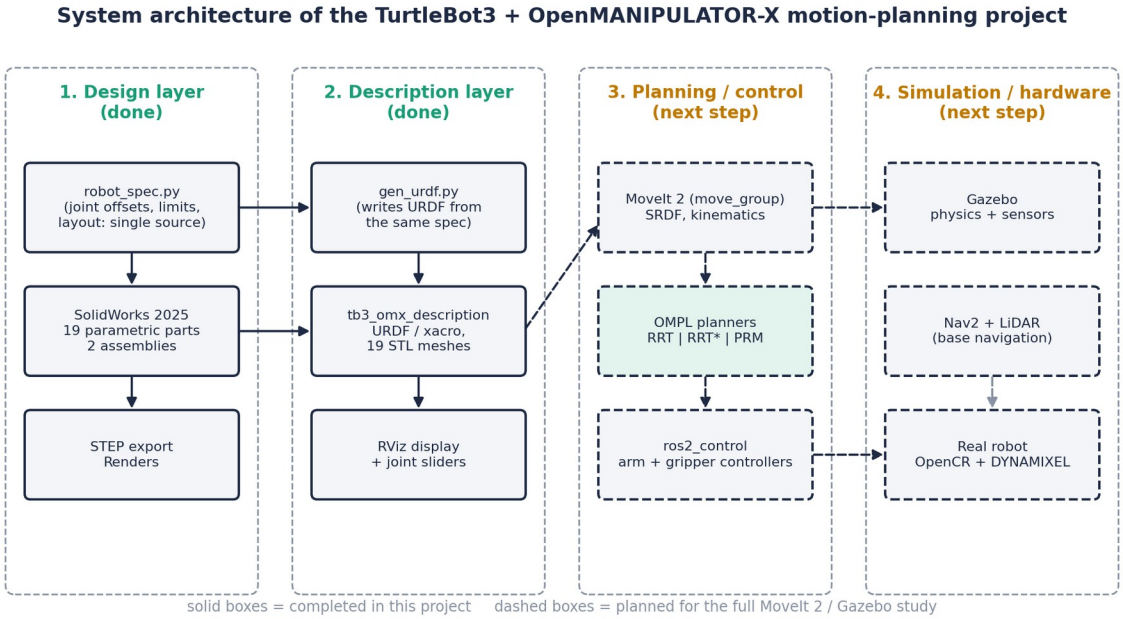


Figure 1. System architecture. Solid boxes are done, dashed boxes are planned.

The algorithm study in this report runs on the pipeline in Figure 2. The planner proposes joint-space configurations. The collision checker turns each one into link positions with forward kinematics and tests them against the obstacles. The planner then returns a path, and we record time, path length, success and node count.

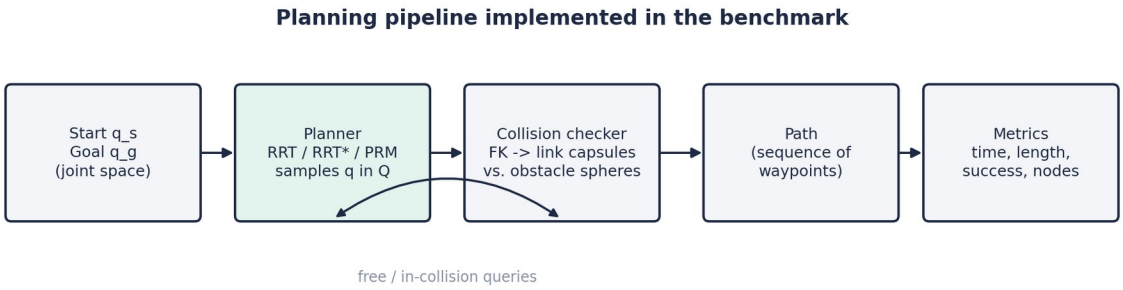


Figure 2. Planning pipeline used in the benchmark.

4.2 CAD modelling

We modelled the robot in SolidWorks 2025, driving the SolidWorks API from Python scripts (tools/build_parts.py and build_robot.py). Every part is parametric: its dimensions are tied to Global Variables (Tools > Equations), for example J3_Z (upper-arm length), J4_X (forearm length), Hole_P (waffle-plate hole pitch) and Tire_D (wheel diameter). The model has 19 parts and 2 assemblies. ASM01 is the Waffle Pi base and ASM00 is the complete robot, with its origin on the ground (base_footprint).

4.3 Assembly and joints

Joints 1 to 4 are mated as real revolute joints (a collinear axis mate plus a coincident plane), and the wheels spin on their axle. The two gripper fingers are slider mates with limits of -10 mm to +19 mm around the 21 mm home position, which gives a jaw opening of roughly 6 mm to 64 mm. Parts are modelled in the ROS convention (x forward, y left, z up), so each link's origin sits exactly on its URDF frame.

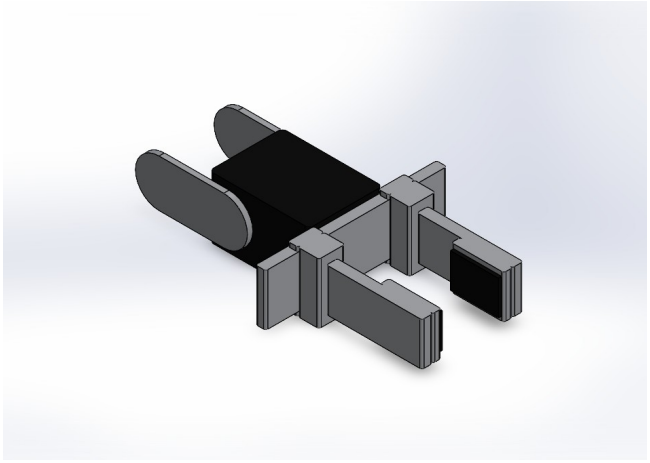


Figure 3. Two-finger gripper on link 5 (SolidWorks render).

4.4 ROS 2 description

tools/gen_urdf.py writes the URDF from the same specification file (robot_spec.py) that drives the CAD, so the two cannot drift apart. Meshes are exported as binary STL in metres. The inertial values are the official ROBOTIS ones. The base and wheels use the official collision primitives, and the arm links use their meshes. Link and joint names are unchanged, so the official SRDF, ompl_planning.yaml, controllers and Nav2 configuration work without edits.

Joint	Parent -> child	Origin (m)	Axis	Limits
base_fixed	base_link -> link1	(-0.092, 0, 0.091)	-	fixed
joint1	link1 -> link2	(0.012, 0, 0.017)	z	+/-0.9 pi
joint2	link2 -> link3	(0, 0, 0.0595)	y	-0.57 pi ... 0.5 pi
joint3	link3 -> link4	(0.024, 0, 0.128)	y	-0.3 pi ... 0.44 pi
joint4	link4 -> link5	(0.124, 0, 0)	y	-0.57 pi ... 0.65 pi
gripper_left/right	link5 -> finger	(0.0817, +/-0.021, 0)	+/-y	-0.010 ... 0.019 m
end_effector_joint	link5 -> tool	(0.126, 0, 0)	-	fixed
wheel_left/right	base_link -> wheel	(0, +/-0.144, 0.023)	z	continuous

Table 2. Kinematic chain, identical to the official ROBOTIS model.

4.5 Verification

The joints (origin, rpy, type and axis) were checked automatically against turtlebot3_manipulation_description from the ROS 2 Humble branch, and the project README records that all of them matched. The package opens in RViz with joint sliders through display.launch.py.

4.6 Mathematical formulation

Forward kinematics

Each arm joint i is placed at offset o_i (Table 2) from its parent and rotates about its axis a_i: the z axis for joint 1 and the y axis for joints 2 to 4. The end-effector pose is the product of these transforms, with the rotations given by Eq. (2).

$$ {}^{0}T_{ee}(\mathbf{q}) = T_{base} \prod_{i=1}^{4} \left[\mathrm{Trans}(\mathbf{o}_i)\, \mathrm{Rot}(\hat{\mathbf{a}}_i,\, q_i) \right] \mathrm{Trans}(\mathbf{o}_{ee}) \tag{1} $$

$$\mathrm{Rot}(\hat{z},q)=\begin{vmatrix}\cos q & -\sin q & 0\\ \sin q & \cos q & 0\\ 0 & 0 & 1\end{vmatrix} \qquad \mathrm{Rot}(\hat{y},q)=\begin{vmatrix}\cos q & 0 & \sin q\\ 0 & 1 & 0\\ -\sin q & 0 & \cos q\end{vmatrix} \tag{2}$$

Configuration space and collision

The planner works in the 4-D joint space Q. A configuration q is free if it respects the joint limits, keeps the upper-arm, forearm and wrist/tool segments (each treated as a capsule of radius r_l = 25 mm) away from every obstacle sphere (centre c_j, radius r_j), and keeps every point from joint 2 onward at least z_min = 20 mm above the base_link plane. The two lowest links and the gripper fingers are not checked in this simplified model. The distance from a link segment to a sphere centre is the point-to-segment distance of Eq. (4). The cost of a path is its joint-space length, Eq. (5).

$$\mathcal{Q}_{free}=\{\mathbf{q}:\ \mathbf{q}_{min}\le\mathbf{q}\le\mathbf{q}_{max},\ \ d\left(\mathrm{link}_i(\mathbf{q}),\,c_j\right)>r_j+r_l\ \ \forall i,j,\ \ z_i(\mathbf{q})\ge z_{min} \tag{3}$$

$$d(\overline{\mathbf{ab}},\mathbf{c})=\min_{t\in[0,1]}\|\,\mathbf{a}+t(\mathbf{b}-\mathbf{a})-\mathbf{c}\,\| \tag{4}$$

$$L(\sigma)=\sum_{k=0}^{m-1}\|\,\mathbf{q}_{k+1}-\mathbf{q}_k\,\|_2 \tag{5}$$

RRT

RRT extends the tree by at most a step eta towards a random sample, Eq. (6). Here q_rand is sampled uniformly inside the joint limits, and with probability 0.1 it is replaced by q_g (goal bias). RRT is probabilistically complete: the probability of finding a path if one exists tends to 1 as the number of samples grows, but it gives no guarantee about path quality.

$$\mathbf{q}_{new}=\mathbf{q}_{near}+\min\left(1,\ \frac{\eta}{\|\mathbf{q}_{rand}-\mathbf{q}_{near}\|}\right)(\mathbf{q}_{rand}-\mathbf{q}_{near}) \tag{6}$$

RRT*

RRT* keeps a cost-to-come c(q) for every node. A new node chooses the parent that minimises cost among its near neighbours, Eq. (7), and then neighbours are rewired through the new node when that lowers their cost, Eq. (8). The neighbour radius shrinks with the number of nodes n, Eq. (9); in our code we use gamma = 2(1 + 1/d)^(1/d) with d = 4, leave out the volume factor, and cap the radius at 1.5 rad (a practical simplification). With this rewiring the path cost converges almost surely to the optimum c*, Eq. (10) [3].

$$\mathbf{q}_{parent} = \arg\min_{\mathbf{q}_j \in \mathcal{Q}_{near}} \left[c(\mathbf{q}_j) + \|\mathbf{q}_j - \mathbf{q}_{new}\| \right] \tag{7}$$

$$c(\mathbf{q}_{new}) + \|\mathbf{q}_{new} - \mathbf{q}_j\| < c(\mathbf{q}_j) \;\Rightarrow\; \text{parent}(\mathbf{q}_j) \leftarrow \mathbf{q}_{new} \tag{8}$$

$$r_n = \min\left\{ \gamma \left(\frac{\ln n}{n}\right)^{1/d},\ \eta \right\}, \qquad \gamma > 2\left(1+\frac{1}{d}\right)^{1/d}\left(\frac{\mu(\mathcal{Q}_{free})}{\zeta_d}\right)^{1/d} \tag{9}$$

$$P\left(\lim_{n\to\infty} c(\sigma_n) = c^*\right) = 1 \quad (\text{RRT}^*) \tag{10}$$

PRM

PRM builds a graph whose vertices are free configurations and whose edges connect each node to its k nearest neighbours when the straight segment between them is free, Eq. (11). A query is then a shortest-path problem solved by Dijkstra's algorithm, Eq. (12). Like RRT it is probabilistically complete, but the path can only follow roadmap edges, so its quality depends on how dense the roadmap is.

$$E = \{(\mathbf{q}_i, \mathbf{q}_j) :\ \mathbf{q}_j \in k\text{NN}(\mathbf{q}_i),\ \overline{\mathbf{q}_i\mathbf{q}_j} \subset \mathcal{Q}_{free}\}, \qquad w_{ij} = \|\mathbf{q}_i - \mathbf{q}_j\| \tag{11}$$

$$D(v) = \min_{u:\,(u,v)\in E} \left[D(u) + w_{uv} \right], \qquad D(\mathbf{q}_s) = 0 \tag{12}$$

Dynamics (how a planned path is executed)

Since this subject is about robot dynamics and control, here is how the planner connects to it. The planner only gives a geometric path. In the full stack that path is given a timing and followed by the

arm controller in ros2_control, and the arm's motion obeys the manipulator equation (13) with inertia matrix M, Coriolis/centrifugal matrix C and gravity vector g. A computed-torque controller, Eq. (14), cancels these terms and makes the tracking error e decay with gains Kp and Kd. We do not simulate the dynamics in this report, but the inertial values in the URDF are the official ROBOTIS ones, so the model is ready for it.

$$\mathbf{M}(\mathbf{q})\,\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\,\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} \tag{13}$$

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\left[\ddot{\mathbf{q}}_d + K_d\,\dot{\mathbf{e}} + K_p\,\mathbf{e}\right] + \mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}), \quad \mathbf{e} = \mathbf{q}_d - \mathbf{q} \tag{14}$$

4.7 The three algorithms step by step

RRT

1. Start the tree T with q_s.
2. Sample q_rand in the joint limits (with probability 0.1 take q_g instead).
3. Find the nearest node q_near in T and step towards q_rand to get q_new (Eq. 6).
4. If the segment q_near -> q_new is collision-free, add q_new to T.
5. If q_new can reach q_g, stop and read the path back through the parents.

RRT*

1. Steps 1-3 as in RRT.
2. Collect the neighbours Q_near within radius r_n (Eq. 9).
3. Choose the parent that gives the lowest cost-to-come (Eq. 7).
4. Rewire: for each neighbour, if going through q_new is cheaper, make q_new its parent (Eq. 8).
5. Do not stop at the first solution; keep sampling until the time budget ends and keep the cheapest path.

PRM

1. Learning phase: sample N collision-free configurations (plus q_s and q_g).
2. Connect each node to its k nearest neighbours if the straight segment is free (Eq. 11).
3. Query phase: run Dijkstra from q_s to q_g over the roadmap (Eq. 12).
4. If no path exists, add more nodes and repeat.

4.8 Benchmark setup

We implemented forward kinematics, the capsule collision check and the three planners in Python (bench.py). The obstacles are three spheres in front of the base: one of radius 50 mm at (0.09, 0, 0.21) m and two of radius 40 mm at (0.10, +/-0.12, 0.14) m. The start pose is q_s = (-1.2, -0.2, 0.6, 0.3) rad and the goal is q_g = (1.2, -0.2, 0.6, 0.3) rad, so joint 1 must swing 2.4 rad. We checked that the straight joint-space line from q_s to q_g is in collision, so the planners have to find a detour; the straight-line length of 2.40 rad is therefore a lower bound for any path. Joint limits and the minimum height are enforced, and each edge is collision-checked every 0.05 rad. RRT uses eta = 0.4 rad, 10 % goal bias and stops at its first solution. RRT* uses the same eta with rewiring and a 10 s time budget. PRM uses 1500 nodes, k = 10 and a Dijkstra query. Each planner ran 10 times with different random seeds. We recorded success rate, time to first solution (for PRM: roadmap construction plus query), joint-space path length and node count.

4.9 Planned MoveIt 2 / Gazebo study

The next step is to repeat the experiment in the full stack with ros2 launch turtlebot3_manipulation_moveit_config moveit_gazebo.launch.py, choosing RRTkConfigDefault, RRTstarkConfigDefault and PRMkConfigDefault in the RViz MotionPlanning panel. That replaces our simplified collision model with the real mesh geometry and Gazebo physics.

5. Results

5.1 CAD model and ROS 2 package

We delivered 19 parametric parts (13 TurtleBot3 parts, TB01-TB13, and 6 OpenMANIPULATOR-X parts, OM01-OM06), two SolidWorks assemblies, one STEP file of the whole robot, and a ROS 2 package with 19 STL meshes. The package contains a standalone URDF (20 links and 19 joints, one of which is the mimic joint for the right gripper finger) and a drop-in xacro that reuses the official Gazebo and ros2_control macros. Figure 4 shows the assembly.

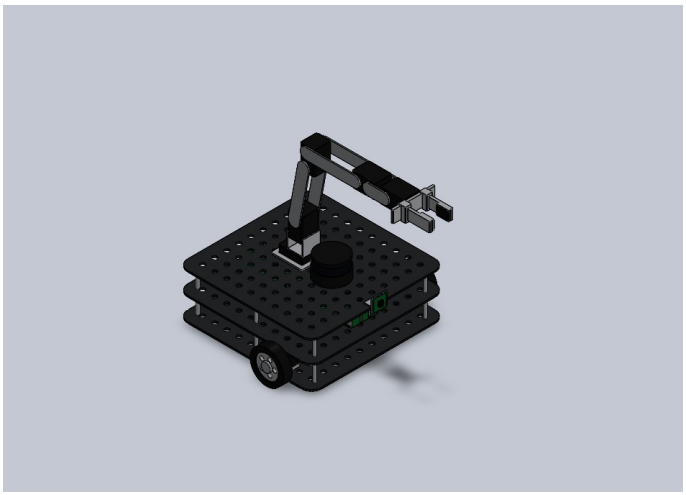


Figure 4. Complete mobile manipulator (isometric).

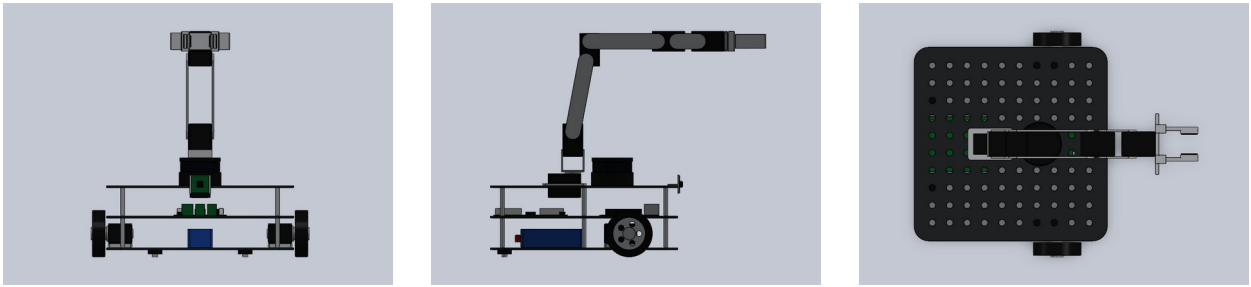


Figure 5. Front, side and top views.

5.2 Arm workspace

Sampling 20 000 random configurations within the joint limits and running forward kinematics gives a maximum end-effector distance of 45.0 cm from the arm's base frame (link1), a maximum height of 55.1 cm above the floor (base_link is 1 cm above the floor, so this is 54.1 cm above base_link) and a maximum forward reach of 28.3 cm in front of the base_link origin (Figure 6). The arm base sits 9.2 cm behind the base_link origin, so the gripper can reach past the front edge of the 266 mm waffle plate.

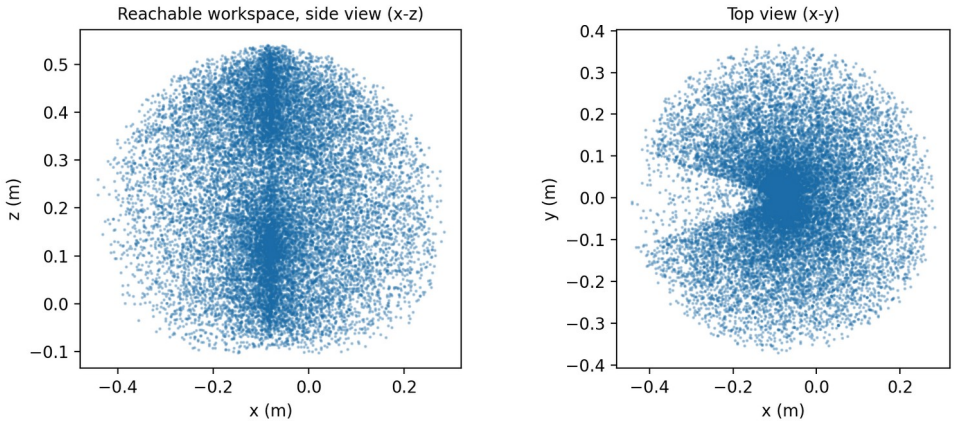


Figure 6. Sampled end-effector workspace.

5.3 Simulation of each algorithm

Note: the planners below are our own Python implementations of the three algorithms, not OMPL, so absolute times depend on our code and the relative trends are what matters. Each figure shows (a) the search structure projected on the q1-q2 plane over a map of colliding configurations (grey, with q3 and q4 fixed at 0.6 and 0.3 rad), (b) arm poses along the returned path in side view, and (c) the four joint angles along the path. In (a) the black dot is the start, the star is the goal and the black line is the path; the path also moves q3 and q4, so it can look as if it crosses the grey area.

5.3.1 RRT

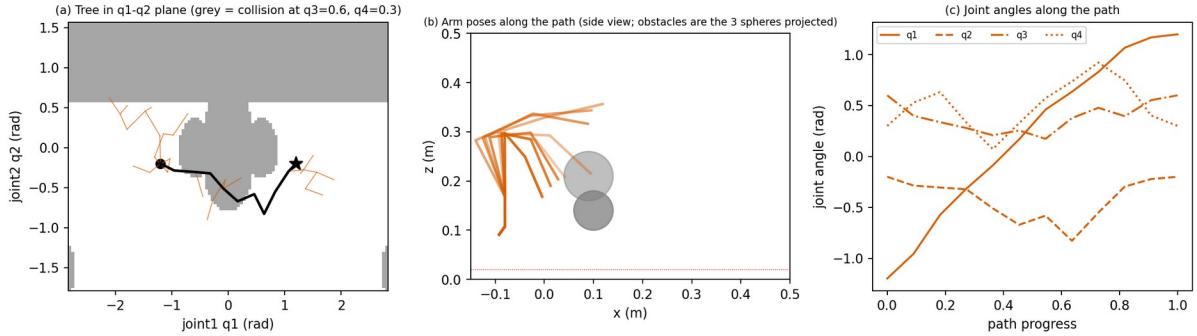


Figure 7. RRT simulation (one run).

RRT reached the goal after growing a tree of only 39 nodes in this run, and returned a path with 12 waypoints. Over 10 runs it needed 43 +/- 12 ms and about 36 nodes.

Justification. RRT always grows from the nearest tree node, which pulls the tree quickly into unexplored space, so it finds a way round the blocking sphere with very few samples. It stops as soon as it reaches the goal and never improves the path, so the route is whatever the random samples happened to give: 4.32 +/- 0.54 rad, which is 80 % longer than the 2.40 rad lower bound and varies a lot between runs. The jagged joint curves in panel (c) show the same effect. That is the speed-versus-quality trade-off: RRT is good for a quick first answer and poor as a final one.

5.3.2 RRT*

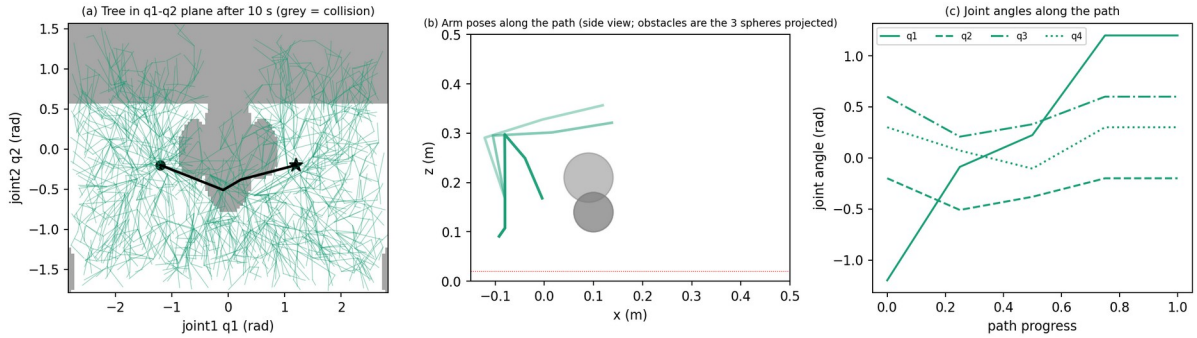


Figure 8. RRT simulation after a 10 s budget (one run).*

RRT* found its first path after about 87 ms, almost as quickly as RRT, and then kept improving it. After 10 s the tree held about 1661 nodes and the path was 2.83 +/- 0.28 rad, only 18 % above the lower bound. The path in this run has 5 waypoints and smooth joint curves.

Justification. Picking the best parent (Eq. 7) and rewiring neighbours (Eq. 8) keeps cutting out detours, so the cost falls over time (Figure 9) and the result is shorter and more repeatable than RRT. The price is computation: each new node looks at all its neighbours, so RRT* builds a tree about 46 times larger than RRT and uses its whole time budget. If time is tight it can be stopped early and still returns a valid path.

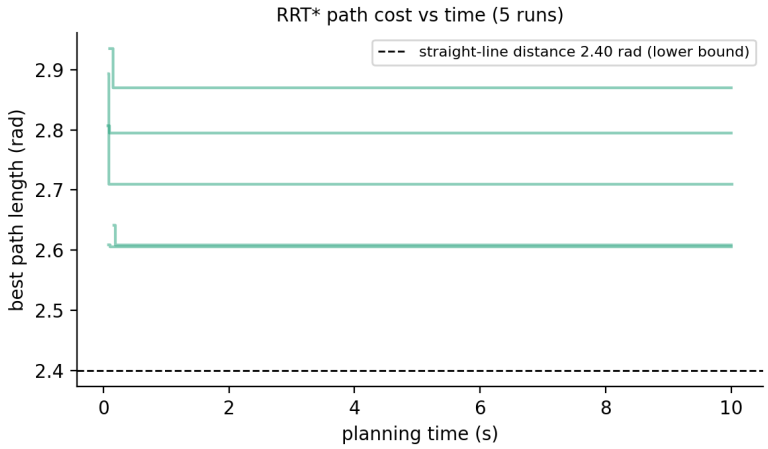


Figure 9. RRT path length against planning time for five runs. Each curve starts when a first path is found.*

5.3.3 PRM

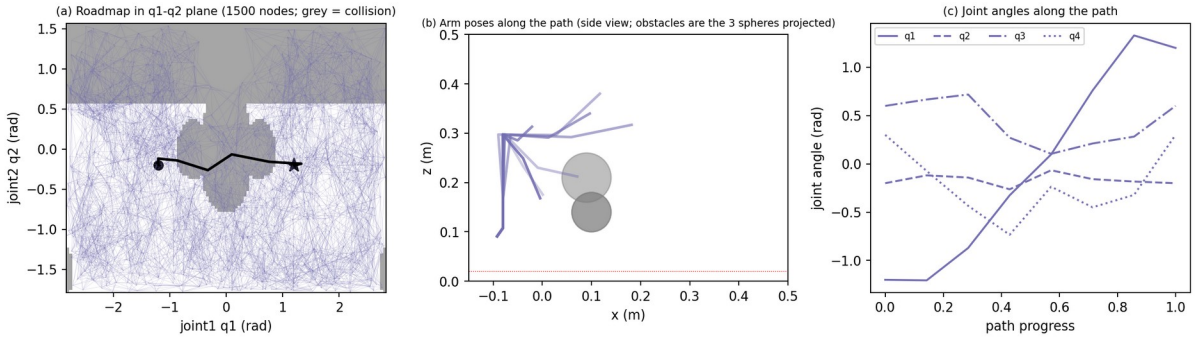


Figure 10. PRM simulation (one run, 1500-node roadmap).

PRM sampled 1500 collision-free nodes and joined them with 7417 edges in this run; Dijkstra then returned a path with 8 waypoints. Over 10 runs it took 10.7 +/- 0.1 s, almost all of it roadmap construction, and returned paths of 3.64 +/- 0.27 rad.

Justification. PRM spends its time up front testing thousands of edges, so the first query is slow, but any later start-goal pair only needs a graph search. The path can only follow edges between stored nodes, so it is shorter than RRT's but cannot match RRT*'s, which keeps refining. Table 3 shows the effect of roadmap size.

Nodes N	Success	Build + query time (s)	Mean path length (rad)
100	10/10	1.4	4.13
250	10/10	2.7	4.22
500	10/10	4.5	3.81
1000	10/10	7.5	3.50
1500	10/10	10.7	3.64

Table 3. PRM roadmap size study (10 runs per size; 1500-node row from the main benchmark). Build time rises steadily with N. Path length tends to fall as N grows (about 4.1-4.2 rad at N = 100-250 against 3.5-3.6 rad at N = 1000-1500), but neighbouring sizes differ by less than the run-to-run spread (about 0.27 rad).

5.4 Comparison

Algorithm	Success	Time to first solution (ms)	Path length (rad)	Above lower bound	Nodes
RRT	10/10	43 +/- 12	4.32 +/- 0.54	80 %	36
RRT*	10/10	87 +/- 64	2.83 +/- 0.28	18 %	1661
PRM	10/10	10694 +/- 138	3.64 +/- 0.27	52 %	1500

Table 4. Results over 10 runs (mean +/- standard deviation). The lower bound is the 2.40 rad straight line, which is blocked. RRT length is the best path after its 10 s budget; PRM time is roadmap construction plus query.*

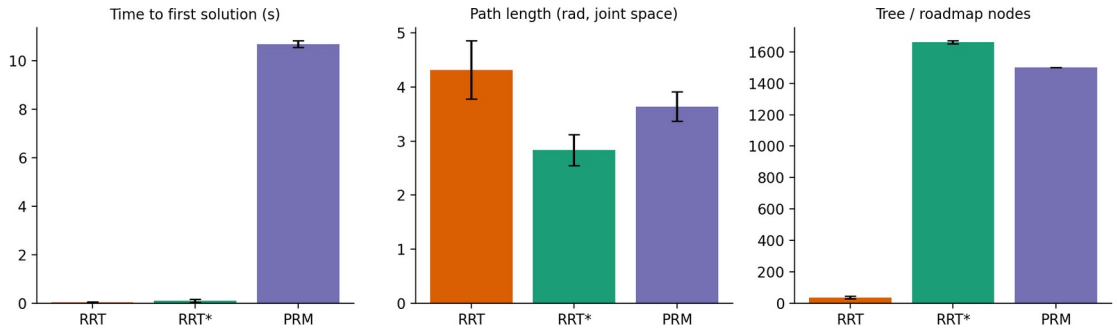


Figure 11. Algorithm comparison.

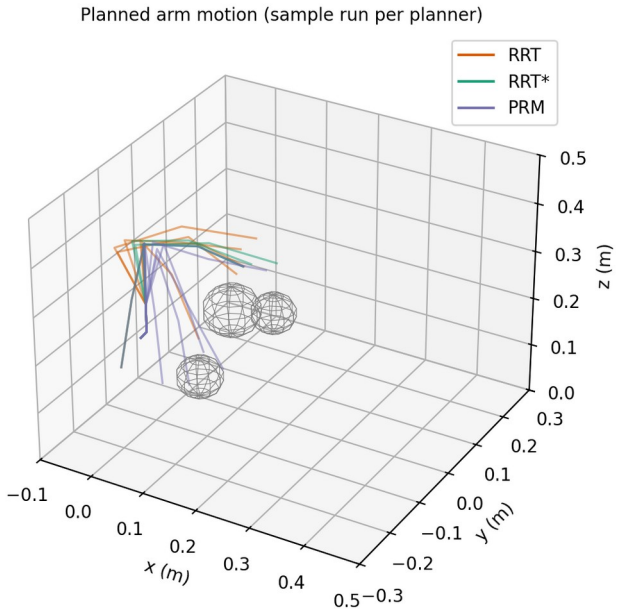


Figure 12. Example arm motions planned by each algorithm around the three obstacles.

5.5 Discussion and justification

All three algorithms solved the problem in every run, as their completeness properties predict. They differ in what it costs to get there: RRT was quickest (43 ms) but gave the longest, least consistent paths; RRT* gave the best paths (2.83 rad, 34 % shorter than RRT) at the price of using its whole time budget; PRM sat in between on quality (3.64 rad) and was by far the slowest for a single query because it first builds the roadmap.

The right algorithm depends on the task. A mobile manipulator keeps arriving at new places, so the scene keeps changing and a one-off planner (RRT or RRT*) fits best; pick RRT* when path quality or smooth joint motion matters. For repeated moves between known poses in a fixed scene, such as a workstation, PRM pays back its set-up cost. So we expect RRT* to be the best default for this robot, and the MoveIt 2 study will check that.

Limitations. Our scene has only three spheres, and the planners are plain Python rather than OMPL, so timings are not comparable with MoveIt numbers. The base is held fixed, and we did not model self-collision, the gripper fingers or the real mesh geometry. In our RRT* the costs of a node's descendants are not updated after a rewire, so its internal cost values are approximate; the reported path lengths are measured on the final path itself and are not affected. Ten runs per planner give only a rough picture of the spread. These limits are why we plan the MoveIt 2 / Gazebo study.

5.6 Conclusion and future work

We produced a parametric CAD and ROS 2 model of the TurtleBot3 + OpenMANIPULATOR-X mobile manipulator whose kinematics match the official one, and we simulated and justified three

planning algorithms on it. The results show the expected trade-off: RRT is quickest, RRT* gives the best and most consistent paths, and PRM pays up front so it can be reused. Future work: (1) run RRTkConfigDefault, RRTstarkConfigDefault and PRMkConfigDefault in MoveIt 2 with Gazebo on cluttered tables and shelves; (2) add RRT-Connect and Informed RRT*; (3) simulate the arm dynamics and a computed-torque controller tracking the planned path; (4) plan base and arm together; (5) test pick-and-place on the physical robot.

6. References

[1] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Tech. Rep. TR 98-11, Computer Science Dept., Iowa State University, 1998.

[2] L. E. Kavraki, P. Svestka, J.-C. Latombe and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," IEEE Trans. Robotics and Automation, vol. 12, no. 4, pp. 566-580, 1996.

[3] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," Int. J. Robotics Research, vol. 30, no. 7, pp. 846-894, 2011.

[4] J. J. Kuffner and S. M. LaValle, "RRT-Connect: An efficient approach to single-query path planning," Proc. IEEE ICRA, pp. 995-1001, 2000.

[5] I. A. Sucan, M. Moll and L. E. Kavraki, "The Open Motion Planning Library," IEEE Robotics & Automation Magazine, vol. 19, no. 4, pp. 72-82, 2012.

[6] S. Chitta, I. Sucan and S. Cousins, "MoveIt!," IEEE Robotics & Automation Magazine, vol. 19, no. 1, pp. 18-19, 2012.

[7] S. Macenski, T. Foote, B. Gerkey, C. Lalancette and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, vol. 7, no. 66, 2022.

[8] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," Proc. IEEE/RSJ IROS, pp. 2149-2154, 2004.

[9] S. Macenski, F. Martin, R. White and J. G. Clavero, "The Marathon 2: A Navigation System," Proc. IEEE/RSJ IROS, 2020.

[10] B. Siciliano, L. Sciavicco, L. Villani and G. Oriolo, Robotics: Modelling, Planning and Control. Springer, 2009.

[11] ROBOTIS Co., Ltd., "TurtleBot3 and OpenMANIPULATOR-X e-Manual," https://emanual.robotis.com (accessed 2026).

[12] ROBOTIS-GIT, "turtlebot3_manipulation" (ROS 2 Humble branch), https://github.com/ROBOTIS-GIT/turtlebot3_manipulation.

[13] J. D. Gammell, S. S. Srinivasa and T. D. Barfoot, "Informed RRT*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," Proc. IEEE/RSJ IROS, pp. 2997-3004, 2014.